InfraFix: Technology-Agnostic Repair of Infrastructure as Code

Nuno Saavedra
nuno.saavedra@tecnico.ulisboa.pt
INESC-ID/IST, University of Lisbon
Lisbon, Portugal

João F. Ferreira
joao@joaoff.com
INESC-ID/IST, University of Lisbon
Lisbon, Portugal

Alexandra Mendes
alexandra@archimendes.com
INESC TEC, Faculty of Engineering,
University of Porto
Porto, Portugal

Abstract

Infrastructure as Code (IaC) enables scalable and automated IT infrastructure management but is prone to errors that can lead to security vulnerabilities, outages, and data loss. While prior research has focused on detecting IaC issues, Automated Program Repair (APR) remains underexplored, largely due to the lack of suitable specifications. In this work, we propose InfraFix, the first technology-agnostic framework for repairing IaC scripts. Unlike prior approaches, InfraFix allows APR techniques to be guided by diverse information sources. Additionally, we introduce a novel approach for generating repair scenarios, enabling large-scale evaluation of APR techniques for IaC. We implement and evaluate InfraFix using an SMT-based repair module and a state inference module that uses system calls, demonstrating its effectiveness across 254,288 repair scenarios with a success rate of 95.7%. Our work provides a foundation for advancing APR in IaC by enabling researchers to experiment with new state inference and repair techniques using InfraFix and to evaluate their approaches at scale with our repair scenario generation method.

CCS Concepts

• **Software and its engineering → Automatic programming**.

Keywords

Infrastructure as Code, Automated Program Repair, DevOps

ACM Reference Format:
Nuno Saavedra, João F. Ferreira, and Alexandra Mendes. 2025. InfraFix: Technology-Agnostic Repair of Infrastructure as Code. In *34th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA Companion '25), June 25–28, 2025, Trondheim, Norway*. ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3713081.3731735

1 Introduction

Infrastructure as Code (IaC) is the process of managing IT infrastructure via programmable configuration files. As IT infrastructure scales, the cost of maintenance and the risk of manual misconfigurations due to human error drastically increases. It becomes impracticable to fully manage IT infrastructures manually and, for that reason, IaC has progressively gained more adoption.

Despite the benefits provided by IaC, such as faster, more reliable, and predictable deployments, errors in these critical pieces of code can lead to security vulnerabilities [25, 28, 29, 31–33], application outages [7, 9, 17, 18, 40], and data losses [43], which undermine the stability and reliability of services and ultimately lead to financial losses or even the loss of human lives. For example, in October 2021, aid workers in conflict zones like Syria were using WhatsApp to communicate about where bombings were taking place, so they could move safely through the country [2]. However, during Facebook's outage, which was triggered by a misconfiguration in their internal backbone network that disconnected Facebook's services and tools [19], the aid workers were unable to communicate.

Given this, it is crucial to understand how we can detect and prevent errors in IaC. Currently, the IaC research community is primarily focused on error detection [3, 4, 15, 21, 24, 25, 29–35, 39, 44]. Only a few studies have taken the additional step of exploring Automated Program Repair (APR) in IaC [14, 42], and most of these efforts focus on static repair rules that do not depend on program-specific specifications [5, 6, 16]. Since most APR techniques use weak specifications, such as test suites [10, 22, 23], the development of APR in IaC is being hindered by the lack of adoption of testing practices for IaC, as noted in prior work [11, 27, 36, 37].

In light of IaC's limited testability, two potential avenues to enable advancements in APR for this domain are: (1) exploring and improving testing practices for IaC — some research has been conducted in this direction [12, 36, 37]; (2) finding different types of specifications for IaC scripts that APR techniques can rely on. We focus our attention on the second avenue. Previous works have presented different types of specifications for IaC [14, 42]. In one of these works, Weiss et al. developed a technique called *imperative configuration repair*, which enables automatic repair of IaC scripts guided by a series of shell commands [42]. Since a common workflow adopted by sysadmins involves using the shell to identify and test required changes to IaC scripts, Weiss et al. used these commands as the specification that guides their APR technique. They implemented their technique in a tool called Tortoise [42].

Building on these ideas, we propose InfraFix, a technology-agnostic framework for repairing IaC scripts. While previous approaches rely on specific information sources, such as shell commands, as specifications, InfraFix is more general by allowing researchers to develop APR techniques guided by specifications derived from diverse information sources. Both the APR technique used and the sources can be adapted by modifying the repair and state inference modules, respectively. Moreover, InfraFix overcomes the limitations of previous APR approaches that were not designed to support multiple IaC technologies simultaneously. According to the 2024 Stack Overflow Developer Survey [41], the IaC landscape is highly fragmented, with adoption rates that vary significantly between technologies. For example, even though Ansible (7.9%) and Terraform (10.6%) are widely used technologies, others

This work is licensed under a Creative Commons Attribution 4.0 International License.
ISSTA Companion '25, Trondheim, Norway
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1474-0/2025/06
https://doi.org/10.1145/3713081.3731735

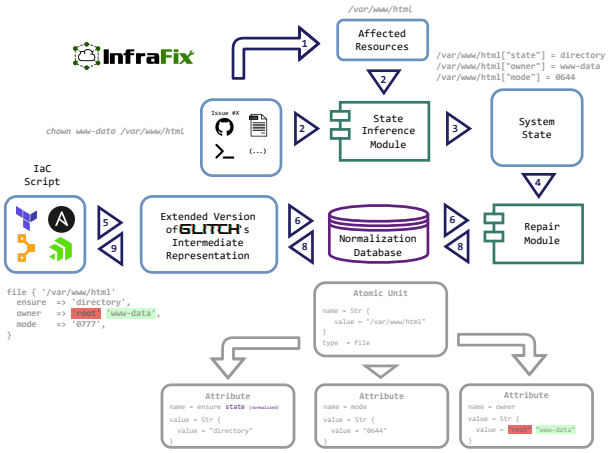

Figure 1: Overview of InfraFix's architecture.

such as Puppet (1.2%), Chef (0.7%) and Nix (2.7%) still have considerable adoption rates. Given this diversity, a technology-agnostic approach is highly desirable.

We make the following contributions: (1) We propose InfraFix, the first technology-agnostic framework that can automatically repair IaC scripts. Our implementation includes an example of an SMT-based repair module and a state inference module. (2) We introduce a novel approach for the generation of repair scenarios that can be used to evaluate APR approaches for IaC. We use this approach to evaluate our implementation of InfraFix in 254,288 different repair scenarios, achieving a successful repair rate of 95.7%.

Tool Availability. A replication package with all the datasets of IaC scripts used to create repair scenarios, the scripts used to run our generation approach, and the source code of the implementation of InfraFix is available at: https://doi.org/10.5281/zenodo.15061764. A video describing and demonstrating InfraFix is available at: https://youtu.be/ETSW0n9DcdU. InfraFix is open-source and available at: https://github.com/sr-lab/GLITCH/tree/issta2025.

2 InfraFix

This section presents InfraFix's technology-agnostic design and modules. Figure 1 shows an overview of InfraFix's architecture. InfraFix starts by taking as input information that describes the desired state of the configured system and then infers the corresponding system state from it (§ 2.1). The framework then receives as input the IaC script to be repaired, which can currently be written in Ansible, Puppet, Chef, or Terraform, and translates it into a normalized intermediate representation (IR) (§ 2.2). Finally, the normalized IR is repaired according to the desired system state, and the changes are propagated back to the original IaC script (§ 2.3).

2.1 State Inference Module

A state inference module creates the specification that guides the repair of the IaC script. This module receives as input information about the system, such as logs or system calls, and a list of resources affected by that information and infers the desired system state. A *system state* is a list of resource states. A resource state consists of the resource's identifier and a dictionary with the attributes of the resource. Due to the declarative nature of IaC scripts, system states are a suitable specification to guide APR techniques. We can use different sources of information to infer states, such as system calls from executed shell commands [42], cloud logs such as AWS CloudTrail or Google Cloud Audit Logs, or GitHub issues.

The current implementation of InfraFix extends Weiss et al.'s approach, providing a state inference module that uses system calls to get a list of files affected by a sysadmin's shell commands. The module then uses that list to retrieve the attributes of those files from the file system. Importantly, InfraFix is flexible and can be easily extended with other valuable approaches. For example, consider the scenario where a sysadmin is asked to install Steam[1] on all Linux machines in a gaming convention. The sysadmin develops an Ansible script that installs the *steam* package on these machines. However, when trying to deploy the script, it returns the following error:

```
Error: You are missing the following 32-bit libraries, and Steam may not run:
libGL.so.1  libdrm.so.2
```

This error is a real-world error that affected many Linux users [20]. Given this error message, a state inference module can produce the following system state:

package:steam { state → present package:libgl1-mesa-dri:i386 { state → present
package:libgl1:i386 { state → present

The inferred system state ensures that the packages required to overcome the error are installed. This system state could guide an APR technique to fix the original Ansible script.

2.2 Technology-Agnostic Approach

To ensure InfraFix is technology-agnostic, we need to address two main challenges: (1) **Syntactic structure:** Different IaC technologies may use different configuration languages. Dealing simultaneously with different configuration languages requires an abstraction that overcomes the different syntactic structures. (2) **Naming:** Depending on the IaC technology, attributes, reserved values, or resource types with the same semantic meaning may have different names. For instance, in Chef, links and directories are managed using distinct resources, whereas in Ansible and Puppet, files, links, and directories are all handled within the same *file* module.

Overcoming these two aspects enables APR techniques to operate consistently across technologies. For example, once the naming is unified, APR can apply the same transformations to the *file* module in Ansible and Puppet as in the *link* and *directory* modules in Chef. Similarly, abstracting control structures such as conditionals and blocks allows to treat them uniformly.

2.2.1 GLITCH's Intermediate Representation. InfraFix addresses the challenge of syntactic differences by leveraging GLITCH's Intermediate Representation (IR). GLITCH is a framework that enables automated technology-agnostic code smell detection for IaC scripts [32, 33]. GLITCH employs an IR into which the scripts of multiple technologies are transformed, and on which analysis techniques are defined to identify code smells. GLITCH's IR follows an object-oriented approach with a hierarchical structure and is able to capture similar concepts from different IaC technologies.

[1] Steam is a video game digital distribution service managed by Valve.

To support APR, we extended GLITCH's IR to support structured expressions. GLITCH's original IR encoded all expressions as a single string. For example, a sum previously represented as the string *"x + y"* is now expressed in our extended IR as a *Sum* node containing two *Variable Reference* nodes. This allows APR techniques to manipulate the values of expressions and pinpoint the location of values in the original IaC script to modify their textual representation. Figure 1 shows a graph-based visualization of how our GLITCH's extended IR models a snippet of an IaC script. For a more detailed description of GLITCH's IR, we refer to GLITCH's original paper by Saavedra and Ferreira [32].

2.2.2 Normalization Database. InfraFix uses a database to normalize resource types, attribute names, and reserved attribute values based on their semantics. If two types of resources in different technologies exhibit the same behavior, they will be mapped to the same type, meaning they will share the same string in the normalized intermediate representation. The same happens between two semantically equivalent pairs of attribute names and values. For example, in Figure 1, the attribute *ensure* is normalized to *state*, its equivalent in Ansible. The chosen normalization names are arbitrary, but must remain consistent across technologies.

The normalization database reduces the cost of supporting semantically equivalent functionality across different technologies. For deterministic APR techniques, it eliminates code duplication. For machine learning APR techniques, training on the normalized IR enables models to possibly generalize across supported technologies. It also expands the available data, allowing scripts from multiple technologies to be used for both training and evaluation.

While only standard system resources may seem to justify normalization, common configuration needs across IaC tools create natural overlaps, making the normalization process broadly applicable. For example, the *community.aws.lightsail* module in Ansible [1] overlaps significantly with the *aws_lightsail_instance* resource in Terraform [13].

2.3 Repair Module

The repair module takes as input the normalized IR of an IaC script and a system state, and is responsible for repairing the normalized IR to align with the desired specification. The repair is applied to the normalized IR and then propagated back to the original IaC script. This process first reverses the normalization rules and then uses information about the location of modified components in the IR, namely the line and column positions, to update their textual representation in the original script.

We integrate the SMT-based approach of Weiss et al. [42] into InfraFix, but we extend it to support repairs for other types of resources other than files. We adopt this approach as it represents the state-of-the-art in APR for IaC. Currently, this module supports the repair of the following resources: files, packages, services and users in Ansible, Puppet, and Chef; AWS IAM roles, AWS EC2 instances, and AWS S3 buckets in Terraform and Ansible. We selected these types of resources because they are shared across the supported IaC technologies and rank among the most frequently used, as evidenced by the statistics provided in our replication package. Furthermore, we improved the approach of Weiss et al. by incorporating support for conditional branching, attribute sketching [38], and direct handling of attributes without requiring preprocessing of IaC scripts, thus expanding the spectrum of potential repairs.

Table 1: Dataset metrics.

	Ansible	Puppet	Chef	Terraform	Tortoise
All scripts	108,511	17,311	70,942	41,550	13
Without test and library scripts	99,934	15,725	40,372	40,972	13
Without duplicates	69,524	12,973	19,846	31,687	13
Curated dataset (Parsable by InfraFix)	64,011	12,753	19,716	30,566	13
Benchmark	12,358	4,868	3,568	1,300	13

3 Evaluation

In this section, we describe the datasets used to evaluate InfraFix, introduce our automated repair scenario generation method, and assess InfraFix's effectiveness in repairing IaC scripts.

3.1 Datasets

To evaluate InfraFix, we use the Ansible and Chef datasets provided by Saavedra and Ferreira [32], the Puppet datasets provided by Rahman et al. [28], and the Terraform dataset provided by Gonçalves [8]. We use these datasets since they have been previously used for large-scale empirical studies on IaC. We also included in our evaluation the Puppet files used to evaluate Tortoise [42]. Subsequently, we curated the IaC scripts in the selected datasets based on the following criteria: (1) Test or library scripts are discarded. (2) If two files contain the same content, only one is kept. (3) Files that are not parsable by InfraFix due to errors are discarded. After applying all the criteria, we obtain five curated datasets. To obtain our evaluation benchmarks, we filter out all files that do not contain at least one resource currently supported by InfraFix. Table 1 shows the number of scripts in the datasets after each step.

3.2 Automated Generation of Repair Scenarios

To perform a comprehensive evaluation of InfraFix, we need to generate repair scenarios for each file in our benchmarks. Manually generating repair scenarios is time-consuming, rendering it unfeasible for large-scale empirical evaluations. For this reason, we created a novel method to automatically generate them. Figure 2 shows an overview of our method.

First, we transform the IaC script into the normalized GLITCH's IR. Then, we employ a state inference module that takes the IR as input to determine the possible system state(s) resulting from applying the original IaC script. We note that multiple potential system states may arise when it is not possible to statically determine which branch of a conditional statement will be executed. After obtaining the possible system states, we apply a sequence of mutations to each state, obtaining a set of desired system states. These mutations simulate potentially desired modifications to the system state. Mutations involve modifying the attributes of the resources in the state of the system with arbitrary yet valid values.

Each desired system state corresponds to a repair scenario. For each scenario, we execute InfraFix with the normalized IR and the desired system state. Since InfraFix can generate multiple possible solutions, we consider the repair successful if any of the fixed GLITCH's IR results in a system state equivalent to the desired

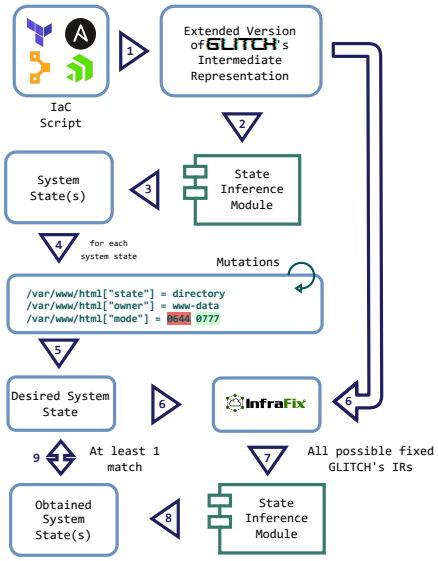

Figure 2: Overview of the method for automated evaluation of APR approaches for IaC.

Table 2: Repair scenarios' results for each benchmark.

	Total	Passed (%)	Failed (%)	Error (%)	Timeout (%)
Ansible	183,825	176,121 (95.8 %)	256 (0.1 %)	155 (0.1 %)	7,293 (4.0 %)
Puppet	4,401	3,497 (79.5 %)	852 (19.4 %)	0 (0.0 %)	52 (1.2 %)
Chef	61,357	59,181 (96.5 %)	1,130 (1.8 %)	64 (0.1%)	982 (1.6 %)
Terraform	3,544	3,544 (100.0 %)	0 (0.0%)	0 (0.0%)	0 (0.0%)
Tortoise	1,161	1,064 (91.7%)	80 (6.9 %)	0 (0.0%)	17 (1.5%)
Total	254,288	243,407 (95.7%)	2,318 (0.9%)	219 (0.1%)	8,344 (3.3 %)

one. If InfraFix raises an error, the scenario is marked as erroneous. If there is a timeout without finding any solutions, we consider the repair scenario to be timed out. Otherwise, we consider it failed. The timeout threshold is set to 120 seconds.

3.3 Results

We successfully generated repair scenarios for 7,833/12,358 (Ansible), 125/4,868 (Puppet), 2,069/3,568 (Chef), 736/1,300 (Terraform), and 13/13 (Tortoise) files. Failures were due to two main reasons: (1) although these files contained supported resources, their names could not be statically resolved to strings; and (2) the resources were defined within other constructs, such as function calls or classes, that were not invoked or instantiated within the same script. As a result, these resources could not be included in the inferred system state for those files.

We generated 183,825 repair scenarios for Ansible, 4,868 for Puppet, 61,357 for Chef, 3,544 for Terraform, and 1,161 for Tortoise. The experiments were conducted on a machine with an Intel Xeon Silver 4110 CPU @ 2.10GHz and 64GB of RAM. We used 32 cores to parallelize the evaluation with all repair scenarios of a single file being handled by the same process. Table 2 shows the results obtained for each benchmark. InfraFix passed 95.8%, 79.5%, 96.5%, 100.0%, and 91.4% of the repair scenarios in the Ansible, Puppet, Chef, Terraform, and Tortoise benchmarks, respectively. Upon manually examining the repair scenarios, we found most failures were due to: (1) references to undefined variables assigned to attributes or other variables, (2) attributes containing shared variable references that, when modified, unintentionally affect multiple attributes, and (3) defining multiple resources within the same construct using an array as title [26].

We do not compare with Tortoise, as it is deprecated due to missing dependencies. We contacted the first and second authors of Tortoise [42], who confirmed that these dependencies are no longer available, making it impossible to compare Tortoise with new APR tools and techniques for IaC. Future work in this area requires a new state-of-the-art tool, for which we propose InfraFix.

4 Related Work

Due to space constraints, we focus on repair techniques specific to IaC. Bui et al. [5] and Durieux et al. [6] developed tools for automated detection and repair of code smells in Dockerfiles. These works are similar to ours in their attempt to repair IaC scripts. However, they focus on code smells, while we consider a wider range of issues. Henkel et al. used machine learning to infer repair rules based on build log analysis. They developed a tool, called Shipwright, with the inferred rules to repair Dockerfiles that fail to build [16]. The main difference is that Shipwright uses pre-defined and inferred rules, while InfraFix infers system state to guide repairs. Hassan et al. proposed RUDSEA, an approach that automatically recommends updates to Dockerfiles based on changes in the source code and build configuration files [14]. Weiss et al. proposed Tortoise, a tool that implements a technique called *imperative configuration repair*, which enables automatic repair of IaC scripts guided by a series of shell commands [42]. Similarly to RUDSEA, Tortoise is focused on a single IaC technology. Our approach is more general, both in terms of resources and IaC technologies supported. In fact, the specifications used by RUDSEA could be used with InfraFix, provided that a state inference module is implemented.

5 Conclusion

We introduce InfraFix, the first technology-agnostic framework for APR in IaC. Beyond supporting multiple IaC technologies, InfraFix enables researchers to explore diverse state inference and repair techniques. We also propose a novel method for generating repair scenarios that facilitate large-scale evaluation of APR solutions for IaC. To demonstrate InfraFix's capabilities, we implement an SMT-based repair module and a state inference module that uses system calls. Our evaluation shows that InfraFix effectively repairs IaC scripts across multiple scenarios. By providing a flexible APR framework and a scalable evaluation methodology, our work establishes a foundation for future advancements in IaC repair. Future work will focus on seamlessly integrating it into sysadmins' workflows by enabling it to operate in the background, automatically suggesting fixes, for instance, through pull requests. We will also extend InfraFix with more repair and state inference modules, and support for additional technologies and resource types.

Acknowledgments

We would like to thank Margarida Ferreira for her help with string encoding, Aaron Weiss and Arjun Guha for their assistance with Tortoise, Vasco Manquinho and Inês Lynce for their feedback, and Bruno Saavedra for his help with the video. This work was supported by Fundação para a Ciência e a Tecnologia (FCT): N. Saavedra by grant BD/04736/2023; N. Saavedra and J. F. Ferreira by projects UIDB/50021/2020 (DOI: 10.54499/UIDB/50021/2020) and the 'InfraGov' project, with ref. n. 2024.07411.IACDC (DOI: 10.54499/2024.07411.IACDC), funded by the 'Plano de Recuperação e Resiliência (PRR)' under the investment 'RE-C05-i08 - Ciência Mais Digital', measure 'RE-C05-i08.M04' (in accordance with the FCT Notice No. 04/C05 i08/2024), framed within the financing agreement signed between the 'Estrutura de Missão Recuperar Portugal (EMRP)' and the FCT as an intermediary beneficiary; A. Mendes was financed by National Funds through the Portuguese funding agency, FCT, within project LA/P/0063/2020 (DOI: 10.54499/LA/P/0063/2020).

References

[1] Ansible project contributors. 2025. community.aws.lightsail module. https://docs.ansible.com/ansible/latest/collections/community/aws/lightsail_module.html [Accessed 20-03-2025].

[2] Avi Asher-Schapiro and Fabio Teixeira. 2021. Facebook down: What the outage meant for the developing world. https://news.trust.org/item/20211005204816-qzjft/ [Accessed 04-09-2024].

[3] Mahi Begoug, Moataz Chouchen, Ali Ouni, Eman Abdullah Alomar, and Mohamed Wiem Mkaouer. 2024. Fine-grained just-in-time defect prediction at the block level in Infrastructure-as-Code (IaC). In *Proceedings of the 21st International Conference on Mining Software Repositories*. 100–112.

[4] Narjes Bessghaier, Mahi Begoug, Chemseddine Mebarki, Ali Ouni, Mohammed Sayagh, and Mohamed Wiem Mkaouer. 2024. On the prevalence, co-occurrence, and impact of infrastructure-as-code smells. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 23–34.

[5] Quang-Cuong Bui, Malte Laukötter, and Riccardo Scandariato. 2023. DockerCleaner: Automatic Repair of Security Smells in Dockerfiles. In *2023 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 160–170.

[6] Thomas Durieux. 2024. Empirical study of the docker smells impact on the image size. In *Proceedings of the IEEE/ACM 46th Int. Conf. on Software Engineering*. 1–12.

[7] GitLab. 2014. Gitlab.com Downtime 2014-07-07 Postmortem. https://docs.google.com/document/d/1ScqXAdb6BjhsDzCo3qdPYbt1uULzgZqPO8zHeHHarS0 [Accessed 04-09-2024].

[8] João Gonçalves. 2023. *Automatic Detection of Security Smells in Terraform*. Master's thesis. Instituto Superior Técnico. Available at https://fenix.tecnico.ulisboa.pt/cursos/meic-a/dissertacao/1972678479055768.

[9] gooeyblob. 2016. Why Reddit was down on Aug 11. https://web.archive.org/web/20221029203405/https://www.reddit.com/r/announcements/comments/4y0m56/why_reddit_was_down_on_aug_11/ [Accessed 04-09-2024].

[10] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. 2019. Automated program repair. *Commun. ACM* 62, 12 (2019), 56–65.

[11] Michele Guerriero, Martin Garriga, Damian A Tamburri, and Fabio Palomba. 2019. Adoption, support, and challenges of infrastructure-as-code: Insights from industry. In *2019 IEEE Int. Conf. on Software Maintenance and Evolution*. 580–589.

[12] Mohammed Mehedi Hasan, Farzana Ahamed Bhuiyan, and Akond Rahman. 2020. Testing practices for infrastructure as code. In *Proceedings of the 1st ACM SIGSOFT International Workshop on Languages and Tools for Next-Generation Testing*. 7–12.

[13] HashiCorp. 2025. Resource: aws_lightsail_instance. https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/lightsail_instance [Accessed 20-03-2025].

[14] Foyzul Hassan, Rodney Rodriguez, and Xiaoyin Wang. 2018. Rudsea: recommending updates of dockerfiles via software environment analysis. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*.

[15] Md Mahadi Hassan, John Salvador, Shubhra Kanti Karmaker Santu, and Akond Rahman. 2024. State Reconciliation Defects in Infrastructure as Code. *Proceedings of the ACM on Software Engineering* 1, FSE (2024).

[16] Jordan Henkel, Denini Silva, Leopoldo Teixeira, Marcelo d'Amorim, and Thomas Reps. 2021. Shipwright: A human-in-the-loop system for dockerfile repair. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*.

[17] Honeycomb. 2021. Incident Resolution: Do You Remember, the Twenty Fires of September? https://www.honeycomb.io/blog/incident-resolution-september-retrospective [Accessed 04-09-2024].

[18] Jan David Nose. 2023. DNS Outage on 2023-01-25. https://blog.rust-lang.org/inside-rust/2023/02/08/dns-outage-portmortem.html [Accessed 04-09-2024].

[19] Santosh Janardhan. 2021. More details about the October 4 outage. https://engineering.fb.com/2021/10/05/networking-traffic/outage-details/

[20] Khantanjil. 2025. Error: You are missing the following 32-bit libraries, and Steam may not run: libGL.so.1 libdrm.so.2. https://github.com/ValveSoftware/steam-for-linux/issues/7284 [Accessed 21-03-2025].

[21] Julien Lepiller, Ruzica Piskac, Martin Schäf, and Mark Santolucito. 2021. Analyzing infrastructure as code to prevent intra-update sniping vulnerabilities. In *27th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer, 105–123.

[22] Martin Monperrus. 2018. Automatic software repair: A bibliography. *ACM Computing Surveys (CSUR)* 51, 1 (2018), 1–24.

[23] Martin Monperrus. 2018. *The Living Review on Automated Program Repair*. Technical Report hal-01956501. HAL/archives-ouvertes.fr.

[24] Ruben Opdebeeck, Ahmed Zerouali, and Coen De Roover. 2022. Smelly variables in ansible infrastructure code: Detection, prevalence, and lifetime. In *Proceedings of the 19th International Conference on Mining Software Repositories*. 61–72.

[25] Ruben Opdebeeck, Ahmed Zerouali, and Coen De Roover. 2023. Control and data flow in security smell detection for infrastructure as code: Is it worth the effort?. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories*.

[26] Puppet, Inc. 2025. Puppet Documentation - Resources: Arrays of Titles. https://www.puppet.com/docs/puppet/latest/lang_resources.html#lang_resource_syntax-arrays-of-titles [Accessed 21-03-2025].

[27] Akond Rahman, Rezvan Mahdavi-Hezaveh, and Laurie Williams. 2019. A systematic mapping study of infrastructure as code research. *Information and Software Technology* 108 (2019), 65–77.

[28] Akond Rahman, Chris Parnin, and Laurie Williams. 2019. The seven sins: Security smells in infrastructure as code scripts. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 164–175.

[29] Akond Rahman, Md Rayhanur Rahman, Chris Parnin, and Laurie Williams. 2021. Security smells in ansible and chef scripts: A replication study. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 30, 1 (2021), 1–31.

[30] Akond Rahman, Shazibul Islam Shamim, Dibyendu Brinto Bose, and Rahul Pandita. 2023. Security misconfigurations in open source kubernetes manifests: An empirical study. *ACM Transactions on Software Engineering and Methodology* 32, 4 (2023), 1–36.

[31] Sofia Reis, Rui Abreu, Marcelo d'Amorim, and Daniel Fortunato. 2022. Leveraging practitioners' feedback to improve a security linter. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–12.

[32] Nuno Saavedra and João F. Ferreira. 2022. GLITCH: Automated Polyglot Security Smell Detection in Infrastructure as Code. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*.

[33] Nuno Saavedra, João Gonçalves, Miguel Henriques, João F. Ferreira, and Alexandra Mendes. 2023. Polyglot Code Smell Detection for Infrastructure as Code with GLITCH. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*.

[34] Julian Schwarz, Andreas Steffens, and Horst Lichter. 2018. Code smells in infrastructure as code. In *2018 11th International Conference on the Quality of Information and Communications Technology (QUATIC)*. IEEE, 220–228.

[35] Tushar Sharma, Marios Fragkoulis, and Diomidis Spinellis. 2016. Does your configuration code smell?. In *Proceedings of the 13th international conference on mining software repositories*. 189–200.

[36] Daniel Sokolowski, David Spielmann, and Guido Salvaneschi. 2024. Automated Infrastructure as Code Program Testing. *IEEE TSE* (2024).

[37] Daniel Sokolowski, David Spielmann, and Guido Salvaneschi. 2024. The PIPr dataset of public infrastructure as code programs. In *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*. IEEE, 498–503.

[38] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. 2006. Combinatorial sketching for finite programs. In *ASPLOS*. 404–415.

[39] Thodoris Sotiropoulos, Dimitris Mitropoulos, and Diomidis Spinellis. 2020. Practical fault detection in puppet programs. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 26–37.

[40] Stack Exchange Network. 2014. Outage Post-Mortem: August 25th, 2014. https://web.archive.org/web/20201020103424/https://stackstatus.net/post/96025967369/outage-post-mortem-august-25th-2014 [Accessed 04-09-2024].

[41] Stack Overflow. 2024. Stack Overflow Developer Survey 2024. https://survey.stackoverflow.co/2024/ [Accessed 03-09-2024].

[42] Aaron Weiss, Arjun Guha, and Yuriy Brun. 2017. Tortoise: Interactive system configuration repair. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 625–636.

[43] Wikimedia. 2017. Incidents/2017-01-18 Labs. https://wikitech.wikimedia.org/wiki/Incidents/2017-01-18_Labs [Accessed 04-09-2024].

[44] Ahmed Zerouali, Ruben Opdebeeck, and Coen De Roover. 2023. Helm charts for Kubernetes applications: Evolution, outdatedness and security risks. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*.